

Word2Vec CBOW in NumPy

Overview

This project implements a simple Word2Vec model in pure NumPy using the CBOW (Continuous Bag of Words) variant with negative sampling. The goal was to build the full optimization loop (forward pass, loss, gradients, and parameter updates) without ML frameworks, and keep the code small and simple.

Dataset

The model is trained on *The White Fang* by Jack London. The text is lowercased and tokenized using a minimal regex-based tokenizer. Rare words are removed with a minimum frequency threshold to keep the vocabulary compact.

Model and Objective

For each target word, a fixed-size context window is taken. CBOW uses the mean of context input vectors as a hidden representation. The objective is the negative sampling loss:

$$\mathcal{L} = -\log \sigma(v_w^\top h) - \sum_{k=1}^K \log \sigma(-v_{n_k}^\top h),$$

where h is the mean context vector, v_w is the output vector of the target word, and v_{n_k} are output vectors of negative samples.

Gradients are derived for the positive term and for each negative sample, and updates are applied to both input and output embeddings.

Training Loop

The training loop iterates over tokens, builds the context, computes the CBOW hidden vector, evaluates loss, and applies gradient updates immediately (online SGD). A running average loss and an accumulated average loss are printed during training for quick feedback.

Optimizations Applied

Only small, safe optimizations were used to keep the code easy to inspect:

- **Vectorized updates for repeated indices:** `np.add.at` is used to update embedding rows when negative samples or context words repeat. This avoids silent errors from overwriting updates and removes small Python loops.
- **Accumulated loss display:** printing both average and accumulated loss makes training progress clearer without extra complexity.
- **Early stopping:** training stops if the epoch average loss does not improve by `min_delta` for `patience` epochs. This prevents unnecessary epochs on a small corpus.

Outputs

The learned input embeddings are saved in a plain text file so they can be inspected or used elsewhere. The script also prints nearest and least similar words as a simple sanity check.